Bayesian Logistic Regression: Predicting Shooting Incidents in Boston

DS 4420 Final Project

Load Libraries

```r
library(ggplot2)
library(dplyr)
```

Load and Prepare Data

```r
df <- read.csv("../data/boston_crime_combined.csv")

# clean up SHOOTING column
df$SHOOTING <- ifelse(df$SHOOTING %in% c(1, "Y"), 1, 0)
df$SHOOTING[is.na(df$SHOOTING)] <- 0

# keep only relevant columns
df <- df[, c("SHOOTING", "MONTH", "HOUR", "DAY_OF_WEEK", "DISTRICT")]

# data is already cleaned in data_prep, just double check
df <- df[complete.cases(df), ]

# encode DAY_OF_WEEK as numeric
day_levels <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday")
df$DAY_OF_WEEK <- as.numeric(factor(df$DAY_OF_WEEK, levels = day_levels))

cat("Size after cleaning:", nrow(df), "\n")
```

```
## Size after cleaning: 899980
```

```r
cat("Any NAs:", anyNA(df), "\n")
```

```
## Any NAs: FALSE
```

```r
cat("Shooting distribution:\n")
```

```
## Shooting distribution:
```

```r
print(table(df$SHOOTING))
```

```
##
##      0      1
## 893278   6702
```

Undersample to Balance Classes

```r
set.seed(42)

shooting_yes <- df[df$SHOOTING == 1, ]
shooting_no  <- df[df$SHOOTING == 0, ]

# randomly sample same number of non-shooting as shooting
shooting_no_sampled <- shooting_no[sample(nrow(shooting_no), nrow(shooting_yes)), ]

df_balanced <- rbind(shooting_yes, shooting_no_sampled)
df_balanced <- df_balanced[sample(nrow(df_balanced)), ]

cat("Balanced dataset size:", nrow(df_balanced), "\n")
```

```
## Balanced dataset size: 13404
```

```r
cat("Shooting distribution after undersampling:\n")
```

```
## Shooting distribution after undersampling:
```

```r
print(table(df_balanced$SHOOTING))
```

```
##
##    0    1
## 6702 6702
```

Build Design Matrix

```r
# one-hot encode DISTRICT
district_dummies <- model.matrix(~ DISTRICT - 1, data = df_balanced)

# combine all features into X matrix
X <- cbind(
  intercept = 1,
  month     = df_balanced$MONTH,
  hour      = df_balanced$HOUR,
  day       = df_balanced$DAY_OF_WEEK,
  district_dummies
)

y <- df_balanced$SHOOTING

cat("Design matrix dimensions:", dim(X), "\n")
```

```
## Design matrix dimensions: 13404 16
```

Train / Test Split

```r
set.seed(42)

n <- nrow(X)
train_idx <- sample(1:n, size = floor(0.8 * n))

X_train <- X[train_idx, ]
```

```r
X_test  <- X[-train_idx, ]
y_train <- y[train_idx]
y_test  <- y[-train_idx]

cat("Train size:", nrow(X_train), "\n")
```

```
## Train size: 10723
```

```r
cat("Test size:", nrow(X_test), "\n")
```

```
## Test size: 2681
```

Bayesian Logistic Regression via Metropolis-Hastings

```r
# sigmoid function
sigmoid <- function(z) 1 / (1 + exp(-z))

# log likelihood of logistic regression
log_likelihood <- function(beta, X, y) {
  p <- sigmoid(X %*% beta)
  p <- pmax(p, 1e-10)
  p <- pmin(p, 1 - 1e-10)
  sum(y * log(p) + (1 - y) * log(1 - p))
}

# log prior: Normal(0, 1) on all coefficients
log_prior <- function(beta) {
  sum(dnorm(beta, mean = 0, sd = 1, log = TRUE))
}

# log posterior = log likelihood + log prior
log_posterior <- function(beta, X, y) {
  log_likelihood(beta, X, y) + log_prior(beta)
}
```

```r
# Metropolis-Hastings sampler
run_mh <- function(X, y, n_iter = 3000, step_size = 0.02) {
  p <- ncol(X)

  beta_current <- rep(0, p)
  lp_current   <- log_posterior(beta_current, X, y)

  samples  <- matrix(0, nrow = n_iter, ncol = p)
  accepted <- 0

  for (i in 1:n_iter) {
    beta_proposed <- beta_current + rnorm(p, sd = step_size)
    lp_proposed   <- log_posterior(beta_proposed, X, y)

    # skip if log posterior is NaN or Inf
    if (is.nan(lp_proposed) || is.infinite(lp_proposed)) {
      samples[i, ] <- beta_current
      next
    }
```

```r
    log_ratio <- lp_proposed - lp_current

    if (log(runif(1)) < log_ratio) {
      beta_current <- beta_proposed
      lp_current   <- lp_proposed
      accepted     <- accepted + 1
    }

    samples[i, ] <- beta_current
  }

  cat("Acceptance rate:", round(accepted / n_iter, 3), "\n")
  colnames(samples) <- colnames(X)
  return(samples)
}
```

```r
set.seed(42)
cat("Running Metropolis-Hastings...\n")
```

```
## Running Metropolis-Hastings...
```

```r
samples <- run_mh(X_train, y_train, n_iter = 3000, step_size = 0.02)
```

```
## Acceptance rate: 0.046
```

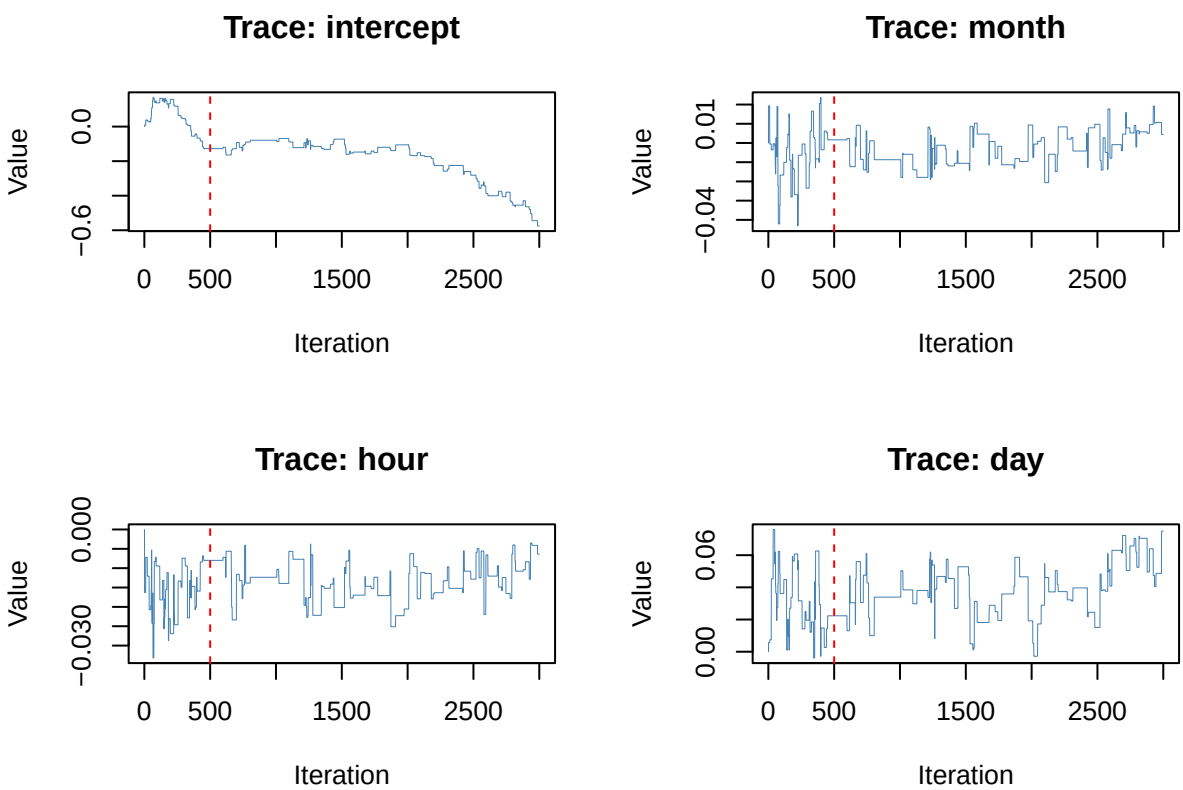
Burn-in and Posterior Samples

```r
# discard first 500 samples as burn-in
burn_in <- 500
post_samples <- samples[(burn_in + 1):nrow(samples), ]

cat("Posterior samples after burn-in:", nrow(post_samples), "\n")
```

```
## Posterior samples after burn-in: 2500
```

Trace Plots

```r
# trace plots for first few coefficients to check convergence
par(mfrow = c(2, 2))

for (i in 1:4) {
  plot(samples[, i], type = "l",
       main = paste("Trace:", colnames(samples)[i]),
       xlab = "Iteration", ylab = "Value",
       col = "steelblue", lwd = 0.5)
  abline(v = burn_in, col = "red", lty = 2)
}
```




```r
par(mfrow = c(1, 1))
```

Posterior Summaries

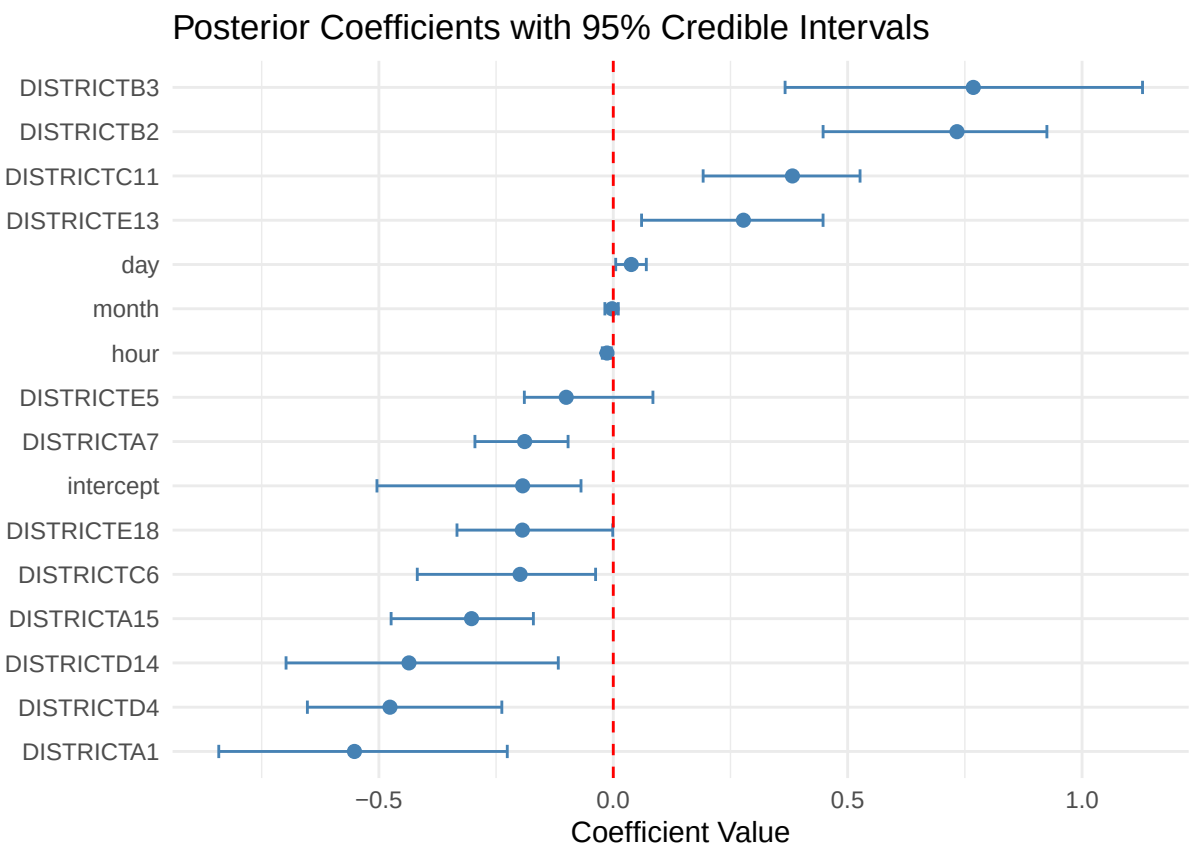
```r
# posterior mean and 95% credible intervals for each coefficient
post_mean <- colMeans(post_samples)
post_ci   <- apply(post_samples, 2, quantile, probs = c(0.025, 0.975))

summary_df <- data.frame(
  coefficient = colnames(post_samples),
  mean        = round(post_mean, 4),
  lower_95    = round(post_ci[1, ], 4),
  upper_95    = round(post_ci[2, ], 4)
)

# flag coefficients whose CI doesn't include 0
summary_df$significant <- ifelse(
  summary_df$lower_95 > 0 | summary_df$upper_95 < 0, "*", ""
)

print(summary_df)
```

```
##             coefficient    mean lower_95 upper_95 significant
## intercept     intercept -0.1933  -0.5041  -0.0688           *
## month             month -0.0027  -0.0179   0.0107
## hour               hour -0.0134  -0.0234  -0.0041           *
## day                 day  0.0384   0.0052   0.0705           *
## DISTRICTA1   DISTRICTA1 -0.5521  -0.8415  -0.2261           *
## DISTRICTA15 DISTRICTA15 -0.3023  -0.4739  -0.1704           *
```

```
## DISTRICTA7   DISTRICTA7 -0.1890  -0.2951  -0.0964          *
## DISTRICTB2   DISTRICTB2  0.7332   0.4476   0.9250          *
## DISTRICTB3   DISTRICTB3  0.7680   0.3665   1.1290          *
## DISTRICTC11 DISTRICTC11  0.3822   0.1917   0.5264          *
## DISTRICTC6   DISTRICTC6 -0.1988  -0.4181  -0.0377          *
## DISTRICTD14 DISTRICTD14 -0.4358  -0.6980  -0.1173          *
## DISTRICTD4   DISTRICTD4 -0.4763  -0.6525  -0.2377          *
## DISTRICTE13 DISTRICTE13  0.2777   0.0604   0.4475          *
## DISTRICTE18 DISTRICTE18 -0.1939  -0.3334  -0.0011          *
## DISTRICTE5   DISTRICTE5 -0.1004  -0.1897   0.0846
```

Posterior Coefficient Plot

```r
ggplot(summary_df, aes(x = mean, y = reorder(coefficient, mean))) +
  geom_point(color = "steelblue", size = 2) +
  geom_errorbarh(aes(xmin = lower_95, xmax = upper_95), height = 0.3, color = "steelblue") +
  geom_vline(xintercept = 0, color = "red", linetype = "dashed") +
  labs(title = "Posterior Coefficients with 95% Credible Intervals",
       x = "Coefficient Value", y = "") +
  theme_minimal()
```



Classification on Test Set

```r
# use posterior mean as point estimate for prediction
beta_hat <- post_mean
```

```r
prob_test <- sigmoid(X_test %*% beta_hat)
pred_test <- ifelse(prob_test > 0.5, 1, 0)

conf_mat <- table(Predicted = pred_test, Actual = y_test)
print(conf_mat)
```

```
##          Actual
## Predicted    0    1
##         0  722  382
##         1  553 1024
```

```r
accuracy <- mean(pred_test == y_test)
cat("Test accuracy:", round(accuracy, 4), "\n")
```

```
## Test accuracy: 0.6512
```

```r
tp <- conf_mat[2, 2]
fp <- conf_mat[2, 1]
fn <- conf_mat[1, 2]

precision <- tp / (tp + fp)
recall    <- tp / (tp + fn)
f1        <- 2 * precision * recall / (precision + recall)

cat("Precision:", round(precision, 4), "\n")
```

```
## Precision: 0.6493
```

```r
cat("Recall:",    round(recall, 4), "\n")
```

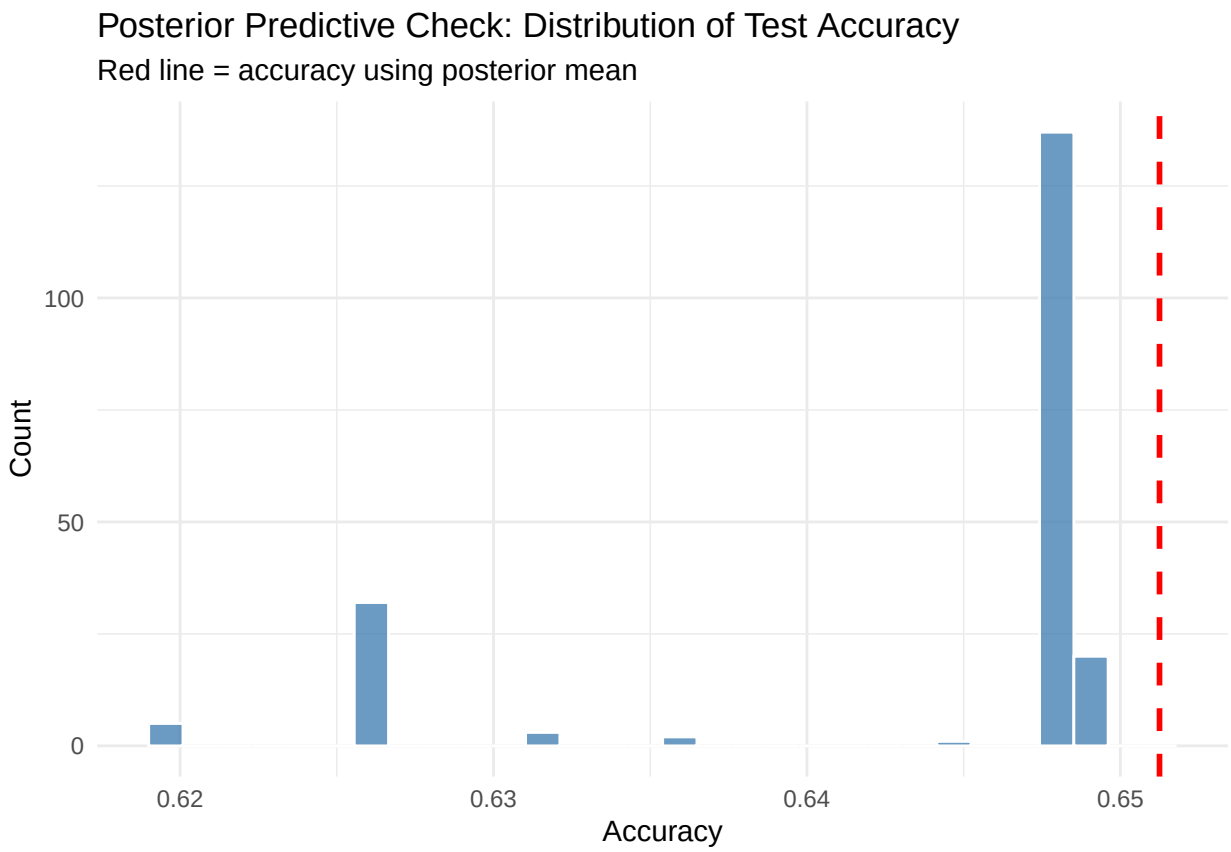
```
## Recall: 0.7283
```

```r
cat("F1 Score:",  round(f1, 4), "\n")
```

```
## F1 Score: 0.6866
```

Posterior Predictive Check

```r
# distribution of test accuracy across posterior samples
n_check <- 200
ppc_acc <- numeric(n_check)

for (i in 1:n_check) {
  b <- post_samples[i, ]
  p <- sigmoid(X_test %*% b)
  preds <- ifelse(p > 0.5, 1, 0)
  ppc_acc[i] <- mean(preds == y_test)
}

ggplot(data.frame(accuracy = ppc_acc), aes(x = accuracy)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white", alpha = 0.8) +
  geom_vline(xintercept = accuracy, color = "red", linetype = "dashed", linewidth = 1) +
  labs(title = "Posterior Predictive Check: Distribution of Test Accuracy",
       subtitle = "Red line = accuracy using posterior mean",
       x = "Accuracy", y = "Count") +
  theme_minimal()
```


Posterior Predictive Check: Distribution of Test Accuracy
Red line = accuracy using posterior mean
Count
100
50
0
0.62
0.63
0.64
0.65
Accuracy